Veri-Store: A Distributed File Storage System with Homomorphic Fingerprint Verification

CS 588 Final Project Report

Mac Payton Albert Huynh
University of Illinois at Chicago

May 2026

---

1 Introduction

Veri-Store is a distributed file storage system that combines erasure coding with homomorphic fingerprint verification to provide fault-tolerant, tamper-evident data storage without requiring full data replication.

The core problem is this: how can a client distribute a data block across multiple servers and, upon retrieval, be certain that the returned data has not been tampered with, even if some servers are actively adversarial? Simple hashing requires the client to store a trusted hash offline; naive replication does not detect subtle corruption.

Veri-Store addresses this with a cryptographic primitive called the fingerprinted cross-checksum (`fpcc`), which allows each fragment to be independently verified using information derived from all fragments at upload time.

Primary Goals

1. **Integrity**: Any byte-level tampering of a stored fragment is detected before the fragment is used in reconstruction.
2. **Fault tolerance**: Up to $n - m$ servers may be unavailable or byzantine; the original block is still recoverable from the remaining m honest servers.
3. **Authenticity**: Each fragment is verifiably bound to the block it originated from; a fragment substitution from a different block is caught.

Veri-Store is explicitly *not* an encryption scheme. Sensitive data should be encrypted before submission. Availability is guaranteed only when at most 2 of 5 servers are lost. (3-of-5 erasure coding, with $m = 3$ and $n = 5$.)

2 System Design

2.1 Erasure Coding: 3-of-5 Reed-Solomon

The client encodes a data block into $n = 5$ fragments using a Cauchy-matrix-based Reed-Solomon scheme over GF(256). Any $m = 3$ of those 5 fragments suffice to reconstruct the original block; the remaining $n - m = 2$ are redundant. This gives availability even when 2 servers are unreachable or corrupt, at the cost of $5/3 \approx 1.67\times$ storage overhead vs. full replication's $5\times$.

Design choice: A Cauchy matrix over GF(256) is used (implemented in `src/erasure/matrix.py`) rather than a Vandermonde matrix, because Cauchy matrices are always invertible for any choice of distinct field elements, simplifying the proof of correctness and eliminating degenerate parameterizations.

2.2 Homomorphic Fingerprinting and the `fpcc`

For a fragment d_i of length L bytes, the fingerprint at evaluation point $r \in \mathrm{GF}(256)$ is:

$$\mathrm{fp}(r, d_i) = \sum_{j=0}^{L-1} d_i[j] \cdot r^j, \quad r, d_i[j] \in \mathrm{GF}(256)$$

This is a polynomial evaluation over GF(256) and is homomorphic: for erasure-coded fragments, $\text{fp}(r, d_0)$ encodes information about all other fragments, so consistency across fragments can be checked without reconstructing the full block.

The fingerprinted cross-checksum (`fpcc`) is generated at upload time and distributed to all servers alongside the fragment and consists of:

- $h_i = \text{SHA-256}(d_i)$ for all $i \in [0, n)$.
- $r = \text{Oracle}(h_0, \ldots, h_{n-1})$ — a deterministic challenge point derived from all fragment hashes via a random-oracle construction.
- $\text{fp}_i = \text{fp}(r, d_i)$ for $i \in [0, m)$.

Design choice: The challenge point r is derived from all fragment hashes, not from a fixed parameter. This means r changes if any fragment changes, binding all fragments together cryptographically without requiring a trusted third party.

2.3 Data Flow

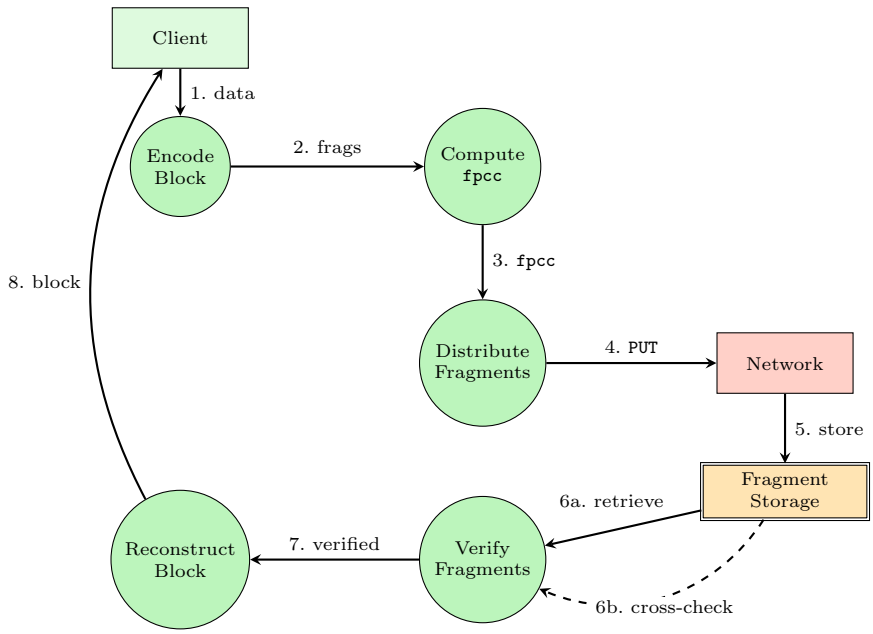

Figure 1: High-level data flow: green = trusted, orange = semi-trusted, red = untrusted.

1. data The client passes the raw data block to the encoder.

2. frags The encoder produces n erasure-coded fragments, passed to fingerprint computation.

3. `fpcc` The `fpcc` (per-fragment hashes and GF(256) fingerprints) is computed across all n fragments.

4. `PUT` Each fragment and the shared `fpcc` are sent to their designated server.

5. store Fragment Storage receives and persists each fragment and `fpcc` to disk.

6a. retrieve On retrieval (`GET`), the server returns the stored fragment and `fpcc` together in a single response.

6b. cross-check The `fpcc` from the same response serves as the integrity reference; the verifier uses it to check each fragment before reconstruction.

7. verified Fragments that pass verification are forwarded to the reconstructor.

8. block The reconstructed original data block is returned to the client.

3 Implementation

3.1 Technology Stack

Component	Library	Purpose
Finite field arithmetic	`src/fingerprint/field.py`	GF(256) polynomial ops
Erasure coding	`src/erasure/`	Cauchy-matrix RS encode/decode
Fragment verification	`src/verification/`	`fpcc` generation; fragment integrity checks
Fragment persistence	`src/storage/`	On-disk fragment and metadata storage
REST API server	`fastapi` ≥ 0.110, `uvicorn`	Fragment storage endpoints
HTTP client	`httpx` ≥ 0.27	Async parallel fragment dispersal
API schema validation	`pydantic` ≥ 2.6	Request/response schema enforcement
Testing	`pytest`, `pytest-asyncio`	Unit and integration tests

Design choice: `fastapi` and `pydantic` were chosen because schema validation happens automatically at the API boundary, rejecting malformed requests (e.g., impossible parameters like $m > n$) before they reach storage logic.

3.2 Module Architecture

```
src/
  erasure/      encoder.py, decoder.py, matrix.py  -- RS encode/decode
  fingerprint/  fingerprint.py, polynomial.py, field.py  -- GF(256) fp
  verification/ cross_checksum.py, oracle.py, verifier.py  -- fpcc + verify
  network/      client.py, server.py, protocol.py, rate_limit.py
  storage/      store.py, fragment.py, metadata.py
```

3.3 Key Protocol: Dispersal (`PUT`)

1. `encoder.py` pads the input to a multiple of m bytes, constructs the Cauchy matrix, and produces n fragments as bytes objects.
2. `cross_checksum.py` computes $\{h_i\}$, derives r via `oracle.py`, and computes $\{\text{fp}_i\}_{i<m}$ to produce the `fpcc`.
3. `client.py` fans out n parallel `httpx PUT` requests, each carrying the fragment (base64-encoded) and the serialized fpcc JSON. A maximum of 3 retries with 0.5s linear backoff handles transient failures.
4. The server-side `server.py` immediately calls `Verifier.check()` on the received fragment before persisting it to disk.

3.4 Key Protocol: Retrieval (`GET`)

1. `client.py` issues n parallel `GET` requests and collects the first m successful responses using `ThreadPoolExecutor`.
2. For each returned fragment, `Verifier.check()` runs two checks:
   (a) `SHA-256` hash against the fpcc entry, and
   (b) for $i < m$, a fingerprint check: $\text{fp}(r', d_i) \stackrel{?}{=}$ `fpcc.fingerprints`$[i]$.
3. Only fragments that return `CONSISTENT` are passed to `decoder.py`, which inverts the Cauchy matrix to recover the original block and strips padding.

4 Security Analysis

4.1 Trust Levels

Administrator: Full access; assumed benign.

Authenticated Client: Verified identity; may run buggy software.

Storage Server: *Semi-trusted* — may be compromised, return stale or corrupted fragments, or attempt to forge

data.

Network: Untrusted; adversary may observe, inject, or modify traffic.

4.2 Core Security Guarantee

Theorem 3.4 (Hendrics et al.) bounds the miss probability of the fingerprint check at $\leq 1/|\mathrm{GF}(256)| = 1/256$ per check. Empirically, testing all single-byte flips across 100 blocks ($\times 5$ fragments each) yielded a 100% detection rate and a 0.0% false positive rate (`docs/experiments/theorem_3_4.md`). The system tolerates up to $n - m = 2$ Byzantine faults: as long as ≥ 3 honest servers are available, the client recovers the original block correctly.

5 System Behavior

5.1 Expected Behavior: Honest Operation

A typical session starts 5 server processes on `localhost:5001..5005`, then uses the client to store and retrieve a file:

```
# Start servers (each on a distinct port)
$ uvicorn src.network.server:app --port 5001 &
...
# Store a file
$ python -m src.network.client store myfile.bin
Dispersed block abc123 to 5/5 servers.  fpcc stored.
# Retrieve the file
$ python -m src.network.client retrieve abc123
Retrieved 3/5 fragments; all verified CONSISTENT.
Reconstructed 4096 bytes -> myfile_out.bin
```

5.2 Expected Behavior: Byzantine Server

When a server returns a corrupted fragment, the verifier rejects it and the client falls back to an honest server:

```
[WARNING] server_2: HASH_MISMATCH for block abc123 fragment 1 (index 1)
[INFO]    Fragment 1 rejected; trying server_4...
[INFO]    Retrieved 3 verified fragments from servers 0, 3, 4.
[INFO]    Reconstructed successfully.
```

5.3 Expected Behavior: Too Many Failures

If more than $n - m = 2$ servers are lost, the client fails closed with a clear error:

```
[ERROR] Only 2 verified fragments available; need 3.  Raising RetrievalError.
RetrievalError: Insufficient verified fragments for block abc123.
```

5.4 Test Suite Results

The security test suite covers verification-layer unit tests, HTTP endpoint tests, and full integration tests wiring a live `VeriStoreClient` to multiple in-process server instances.

- **Unit tests**: all byte-flip mutations on test fragments detected; `HASH_MISMATCH` or `FP_MISMATCH` returned in every case.
- **Endpoint tests**: malformed base64, impossible parameters ($m > n$), duplicate `PUT`, and missing bearer token all return the correct HTTP error codes (400, 409, 422, 401).
- **Integration tests**: end-to-end store/retrieve with 0, 1, and 2 Byzantine servers; failure with 3 Byzantine servers confirmed.
- **Empirical experiments**: collision-probability experiment over 500 checks confirms $\leq 1/256$ theoretical bound; false positive rate = 0.0%.